University of Versailles Saint-Quentin-en-Yvelines
Paris-Saclay University
Master's Degree in DataScale — Large-Scale Data Management

M2 Internship Notes : Graph Neural Networks (GCN, GAT, and GIN)

Prepared by

Fidèle OUEDRAOGO

Supervisors

Prof. Christine Largeron

Dr. Haifei Zhang

Date

7 March 2026

Hubert Curien Laboratory
2025–2026

Contents

Introduction

Relational data are ubiquitous in numerous domains, including social networks, citation networks, recommender systems, biological networks, and transportation systems. Unlike traditional tabular data, they are naturally represented as graphs, where nodes model entities and edges represent the relationships between them. This relational structure makes graph analysis particularly challenging, as meaningful information arises not only from the attributes of individual nodes but also from the topology of the graph itself.

Learning from graph-structured data has become a fundamental problem in machine learning, with applications to several tasks such as node classification, link prediction, graph classification, and community detection. For many years, these tasks relied primarily on manually engineered graph features or shallow graph embedding techniques. Although these approaches achieved promising results in certain applications, they suffer from several limitations, particularly their inability to effectively leverage both node attributes and the structural information of the graph in a unified learning framework.

Graph Neural Networks (GNNs) were introduced to overcome these limitations by learning node and graph representations directly from both feature information and graph topology through neighborhood aggregation. Today, GNNs have become one of the most successful frameworks for graph representation learning and have inspired numerous architectures. Among them, the Graph Convolutional Network (GCN), the Graph Attention Network (GAT), and the Graph Isomorphism Network (GIN) are three of the most influential models and constitute the main focus of this study [1–3].

Chapter 1

Background

1.1 Graph Representation Learning

Graph representation learning aims at designing methods that can transform graph-structured data into meaningful vector representations while preserving both structural and attribute information. A graph is formally defined as $G = (V, E)$, where V is the set of nodes (vertices) and E is the set of edges representing relationships between nodes. Graphs provide a natural way to model relational systems such as social networks, citation networks, biological networks, or recommender systems.

Unlike traditional tabular data, graph data introduces additional complexity due to the dependencies between nodes. This makes learning tasks more challenging, as information is not only contained in individual node features but also in the connectivity pattern of the graph.

Graph learning methods are generally evaluated on several standard tasks, including node classification, link prediction, graph classification, and community detection. These tasks require models capable of capturing both local and global structural patterns.

1.1.1 Traditional Methods

Before the emergence of deep learning approaches on graphs, graph analysis relied heavily on manually designed features and shallow embedding techniques.

Graph Statistics

Early approaches focused on extracting structural statistics from graphs. Common measures include node degree, centrality, and clustering coefficient.

The degree of a node u is defined as:

$$d_u = \sum_v A[u, v]$$

where A is the adjacency matrix of the graph.

Centrality measures, such as eigenvector centrality, capture the importance of a node based on the importance of its neighbors:

$$e_u = \frac{1}{\lambda} \sum_v A[u, v] e_v$$

The clustering coefficient measures how connected the neighbors of a node are:

$$c_u = \frac{\text{number of triangles around } u}{\text{number of possible triangles}}$$

These features are simple but limited, as they are manually engineered and fixed for all tasks.

Graph Laplacian

Another fundamental tool in graph theory is the graph Laplacian, defined as:

$$L = D - A$$

where D is the degree matrix and A is the adjacency matrix.

The Laplacian matrix encodes the structure of the graph and is widely used in spectral clustering. Its eigenvalues and eigenvectors provide important insights into graph connectivity. In particular, the smallest eigenvalues correspond to connected components, while the second smallest eigenvector is used for graph partitioning (min-cut relaxation problem).

Node Embedding Methods

To overcome the limitations of handcrafted features, node embedding methods were introduced to learn continuous vector representations of nodes. The goal is to map each node u to a vector $z_u \in \mathbb{R}^d$ such that structurally similar nodes are close in the embedding space.

Formally, embeddings are learned through an encoder-decoder framework:

Encoder:

$$enc(u) = z_u$$

Decoder:

$$dec(z_u, z_v) \rightarrow similarity(u, v)$$

The objective is to reconstruct a similarity matrix S:

$$L = \sum (dec(z_u, z_v) - S[u, v])^2$$

Methods such as DeepWalk and node2vec define similarity based on random walks:

$$P(v|u)$$

and aim to learn embeddings such that:

$$z_u \cdot z_v \approx P(v|u)$$

These methods are equivalent to implicit matrix factorization of a graph-based co-occurrence matrix.

However, these approaches suffer from important limitations: they do not use node features, do not share parameters across nodes, and are not inductive.

Limitations of Traditional Methods

Despite their effectiveness in certain scenarios, traditional graph learning methods have several limitations:

- handcrafted features lack adaptability,
- shallow embeddings ignore node attributes,
- models are transductive and do not generalize to unseen nodes,
- structural and feature information are not jointly learned.

These limitations motivated the development of Graph Neural Networks.

1.1.2 Graph Neural Networks

Graph Neural Networks (GNNs) were introduced to overcome the limitations of traditional methods by learning representations directly from graph structure and node features through neural architectures.

General Principle

The key idea behind GNNs is to compute node representations by iteratively aggregating information from neighboring nodes. Unlike shallow embeddings, where each node has an independent parameter vector, GNN embeddings are computed dynamically from the graph structure.

This allows the model to capture both local and global dependencies in the graph.

Message Passing Framework

Most GNN architectures follow the message passing paradigm. At each layer k, each node u performs two operations:

Aggregation:

$$m_{N(u)}^{(k)} = \text{AGGREGATE}^{(k)}\left(\{h_v^{(k)} \mid v \in N(u)\}\right)$$

Update:

$$h_u^{(k+1)} = \text{UPDATE}^{(k)}\left(h_u^{(k)}, m_{N(u)}^{(k)}\right)$$

The initial node representations are given by:

$$h_u^{(0)} = x_u$$

After K layers, the final embedding is:

$$z_u = h_u^{(K)}$$

Each layer extends the receptive field of a node: - 1 layer → 1-hop neighbors - 2 layers → 2-hop neighbors - K layers → K-hop neighborhood

Advantages over Traditional Methods

Compared to classical embedding methods, GNNs offer several advantages:

- they jointly learn structure and features,
- they share parameters across nodes,
- they support inductive learning,
- they naturally generalize to unseen graphs or nodes.

Overall, GNNs provide a unified framework for learning on graph-structured data through iterative information propagation between nodes.

Chapter 2

Graph Neural Network Models

2.1 From Basic GNN to Advanced Architectures

2.1.1 Basic Graph Neural Network

Before introducing specialized architectures such as GCN or GAT, it is important to understand the fundamental idea behind Graph Neural Networks (GNNs). A basic GNN defines node representations by iteratively aggregating information from neighboring nodes.

At layer k, the representation of a node u is updated as follows:

$$h_u^{(k)} = \sigma\left(W_{\text{self}}^{(k)} h_u^{(k-1)} + W_{\text{neigh}}^{(k)} \sum_{v \in N(u)} h_v^{(k-1)} + b^{(k)}\right)$$

his formulation shows that each node combines: its own previous representation and the aggregated information from its neighbors.

In matrix form, this can be written as:

$$H^{(k)} = \sigma\left(AH^{(k-1)}W_{\text{neigh}}^{(k)} + H^{(k-1)}W_{\text{self}}^{(k)}\right)$$

A common variant introduces self-loops, allowing each node to explicitly include its own information in the aggregation process:

$$H^{(k)} = \sigma\left((A+I)H^{(k-1)}W^{(k)}\right)$$

where I is the identity matrix.

To ensure numerical stability and avoid exploding representations, a normalization step is often applied:

$$H^{(k)} = \sigma\left(\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}H^{(k-1)}W^{(k)}\right)$$

where $\tilde{A} = A + I$ and $\tilde{D}$ is its degree matrix.

This normalization prevents nodes with high degrees from dominating the aggregation process and improves training stability.

Key idea: A basic GNN updates node representations by combining self-information and neighborhood information through learned transformations and aggregation.

2.1.2 Graph Convolutional Network (GCN)

The Graph Convolutional Network (GCN) is a simplified and well-optimized version of the basic GNN framework. It introduces a symmetric normalization of the adjacency matrix, leading to a more stable and efficient learning process.

The propagation rule of a GCN layer is defined as:

$$H^{(k)} = \sigma\left(\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}H^{(k-1)}W^{(k)}\right)$$

where:

- $\tilde{A} = A + I$ (graph with self-loops),
- $\tilde{D}$ is the degree matrix of $\tilde{A}$,
- $W^{(k)}$ is a learnable weight matrix,
- σ is a nonlinear activation function.

Intuition: Each node aggregates a normalized average of its neighbors and itself, followed by a linear transformation and a non-linear activation.

GCN can be interpreted as:

- message passing,
- self-loop aggregation,
- degree normalization,
- feature transformation.

This combination makes GCN both efficient and effective for semi-supervised node classification tasks.

2.1.3 Graph Attention Network (GAT)

The Graph Attention Network (GAT) extends the idea of GCN by introducing a learnable attention mechanism that assigns different importance weights to different neighbors.

Instead of treating all neighbors equally, GAT computes attention coefficients:

$$\alpha_{u,v} = \frac{\exp\left(a^\top [Wh_u \| Wh_v]\right)}{\sum_{v' \in N(u)} \exp\left(a^\top [Wh_u \| Wh_{v'}]\right)}$$

where:

- W is a shared linear transformation,
- a is a learnable attention vector,
- $\|$ denotes concatenation.

The new representation of node u is then computed as:

$$h_u^{(k)} = \sigma\left(\sum_{v \in N(u)} \alpha_{u,v} W h_v^{(k-1)}\right)$$

Multi-head attention: GAT uses multiple attention heads to stabilize learning and capture different aspects of the neighborhood. Outputs from different heads are concatenated or averaged.

Key difference with GCN:

- GCN: fixed normalization weights based on graph structure,
- GAT: adaptive weights learned through attention.

Advantages:

- better expressiveness,
- adaptive neighbor importance,
- improved performance on heterogeneous neighborhoods.

2.1.4 GraphSAGE

GraphSAGE (Graph Sample and Aggregate) is an inductive Graph Neural Network proposed by Hamilton et al. that learns a function to generate node embeddings by aggregating information from local neighborhoods, instead of learning a unique embedding for each node. This inductive formulation enables GraphSAGE to generalize to previously unseen nodes and even unseen graphs.

Given a node v, GraphSAGE recursively updates its representation through neighborhood aggregation. At each layer k, the neighborhood representations are first aggregated:

$$h_{N(v)}^{(k)} = \text{AGGREGATE}^{(k)}\left(\left\{h_u^{(k-1)},\ \forall u \in N(v)\right\}\right)$$

The aggregated neighborhood representation is then concatenated with the current node representation and transformed as follows:

$$h_v^{(k)} = \sigma\left(W^{(k)} \cdot \text{CONCAT}\left(h_v^{(k-1)}, h_{N(v)}^{(k)}\right)\right)$$

The GraphSAGE pipeline consists of four successive steps:

1. Sample a fixed-size neighborhood,
2. Aggregate neighborhood representations,
3. Concatenate the aggregated neighborhood with the node representation,
4. Apply a linear transformation followed by a non-linear activation.

Aggregator variants:

- **Mean Aggregator**

$$h_{N(v)}^{(k)} = \frac{1}{|N(v)|} \sum_{u \in N(v)} h_u^{(k-1)}$$

- **GCN Aggregator**

$$h_v^{(k)} = \sigma\left(W^{(k)} \cdot \text{MEAN}\left(\{h_v^{(k-1)}\} \cup \{h_u^{(k-1)},\, u \in N(v)\}\right)\right)$$

- **LSTM Aggregator**

$$h_{N(v)}^{(k)} = \text{LSTM}\left(\{h_u^{(k-1)},\, u \in N(v)\}\right)$$

- **Pooling Aggregator**

$$\text{AGGREGATE}_{pool}^{(k)} = \max\left(\left\{\sigma\left(W_{pool} h_u^{(k)} + b\right), \forall u \in N(v)\right\}\right)$$

Key differences:

- Compared with GCN, GraphSAGE is inductive and generates embeddings through neighborhood aggregation, allowing generalization to unseen nodes and graphs.
- Unlike GAT, GraphSAGE does not learn attention weights; neighbors are aggregated using predefined aggregation functions (mean, GCN, LSTM or pooling).

Advantages:

- inductive representation learning,
- scalable to large graphs through neighborhood sampling,
- supports unseen nodes and unseen graphs,
- flexible aggregation architectures.

2.1.5 Graph Isomorphism Network (GIN)

The Graph Isomorphism Network (GIN) is designed to maximize the expressive power of Graph Neural Networks. Unlike previous architectures such as GCN or GAT, GIN is explicitly motivated by theoretical results showing that many GNN variants fail to distinguish certain graph structures, particularly when using mean or max aggregation functions.

In particular, mean and max aggregators may map different neighborhood structures to the same embedding, limiting their ability to capture structural differences. In contrast, the

sum aggregator has stronger discriminative power and can distinguish different multisets of neighbor features.

Based on this observation, GIN adopts a simple but powerful update rule:

$$h_u^{(k)} = \text{MLP}^{(k)} \left((1 + \epsilon^{(k)}) h_u^{(k-1)} + \sum_{v \in N(u)} h_v^{(k-1)} \right)$$

where:

- $\epsilon^{(k)}$ is either a learnable parameter or a fixed scalar,
- $\text{MLP}^{(k)}$ is a multi-layer perceptron,
- the sum aggregation ensures injective neighborhood aggregation under mild conditions.

Key idea: GIN increases the expressive power of GNNs by combining sum aggregation with an MLP, allowing it to better distinguish different graph structures.

To obtain a graph-level representation, GIN aggregates representations from all layers using a readout function:

$$h_G = \text{CONCAT}\left(\text{READOUT}(\{h_v^{(k)} | v \in G\}) \mid k = 0, 1, \ldots, K\right)$$

where the READOUT function typically uses summation over node embeddings.

Intuition: GIN can be interpreted as a GNN that approximates the discriminative power of the Weisfeiler-Lehman graph isomorphism test, making it one of the most expressive architectures among message-passing GNNs [3].

Summary: GIN improves over previous models by:

- using sum aggregation (instead of mean or max),
- employing MLPs instead of linear transformations,
- achieving stronger theoretical expressiveness.

Chapter 3

Experimental Study

3.1 Datasets

3.1.1 Datasets from Original Papers

The following benchmark datasets were used to reproduce the results reported in the original papers.

Table 3.1: Node classification datasets used in GCN, Graphsage and GAT reproduction experiments.

Dataset	Nodes	Edges	Features	Classes
Cora	2708	5278	1433	7
Citeseer	3327	4552	3703	6
PubMed	19717	44324	500	3

These datasets correspond to citation networks where nodes represent scientific publications and edges represent citation links.

For graph classification (GIN experiments), the following datasets were used:

Table 3.2: Graph classification datasets used for GIN experiments (Xu et al.).

Dataset	Graphs	Classes	Avg Nodes
IMDB-BINARY	1000	2	19.8
IMDB-MULTI	1500	3	13.0
REDDIT-B	2000	2	429.6
REDDIT-M5K	5000	5	508.5
COLLAB	5000	3	74.5
MUTAG	188	2	17.9
PROTEINS	1113	2	39.1
PTC	344	2	25.5
NCI1	4110	2	29.8

3.1.2 Other Datasets

In addition to classical benchmarks, several new datasets provided by the supervisors were used to evaluate the generalization capability of GCN, Graphsage and GAT models.
Table 3.3: Additional datasets provided during the internship (heterophily and real-world graphs).

Dataset	Nodes	Edges	Classes	Domain
Cornell	183	280	5	Web pages
Texas	183	295	5	Web pages
Wisconsin	251	466	5	Web pages
Squirrel	5201	198493	5	Wikipedia network
Chameleon	2277	31421	5	Wikipedia network
Actor	7600	26752	5	Co-occurrence network
Roman-Empire	22662	32927	18	Linguistic graph
Amazon-Ratings	24492	93050	5	Product network
Minesweeper	10000	39402	2	Synthetic grid
Tolokers	11758	519000	2	Crowdsourcing network
Questions	48921	153540	2	User interaction network

3.2 Node Signature Used for Strict Duplicate Detection

For each node $v_i \in V$, the signature is defined as

$$\boxed{S(v_i) = (x_i, y_i, \text{sort}(N(i)))}$$

where

- x_i denotes the node feature vector;
- y_i is the node label;
- $N(i)$ is the neighborhood of node v_i;
- $\text{sort}(N(i))$ provides a canonical ordering of the neighbors so that two identical neighborhoods remain identical independently of the edge storage order.

Two nodes are considered strict duplicates iff

$$S(v_i) = S(v_j),$$

that is,

$$x_i = x_j, \qquad y_i = y_j, \qquad \text{sort}(N(i)) = \text{sort}(N(j)).$$

Only one representative node is kept in each duplicate group.

3.3 Experimental Results

3.3.1 Squirrel

The duplicate detection algorithm identified 16 duplicate groups, resulting in the removal of 21 nodes.

Table 3.4: Statistics before and after duplicate removal on Squirrel

Statistic	Before	After
Number of nodes	5201	5180
Number of edges	217073	212725
Node features	2089	2089
Classes	5	5
Average degree	83.4736	82.1332
Class 0	1042	1028
Class 1	1040	1038
Class 2	1039	1034
Class 3	1040	1040
Class 4	1040	1040

The preprocessing removed

$$21 \text{ nodes } (0.40\%)$$

and

$$4348 \text{ edges.}$$

3.3.2 Chameleon

The duplicate detection algorithm identified 62 duplicate groups, resulting in the removal of 165 nodes.

Table 3.5: Statistics before and after duplicate removal on Chameleon

Statistic	Before	After
Number of nodes	2277	2112
Number of edges	36101	33114
Node features	2325	2325
Classes	5	5
Average degree	31.7093	31.3580
Class 0	456	370
Class 1	460	390
Class 2	453	445
Class 3	521	520
Class 4	387	387

The preprocessing removed

$$165 \text{ nodes } (7.25\%)$$

and

$$2987 \text{ edges.}$$

3.4 Evaluation Metrics

The performance of all models is evaluated using three complementary metrics: accuracy, Macro-F1 score, and Micro-F1 score.

3.4.1 Accuracy

Accuracy measures the proportion of correctly classified samples over the total number of samples:

$$\text{Accuracy} = \frac{\text{TP}+\text{TN}}{\text{TP}+\text{FP}+\text{TN}+\text{FN}}$$

3.4.2 Macro-F1 Score

The Macro-F1 score computes the F1-score independently for each class and then averages the results, giving equal importance to all classes regardless of their sizes.

$$\text{Macro-F1} = \frac{1}{C} \sum_{i=1}^{C} F1_i$$

where C is the number of classes and $F1_i$ denotes the F1-score of class i.

3.4.3 Micro-F1 Score

The Micro-F1 score aggregates the true positives (TP), false positives (FP), and false negatives (FN) across all classes before computing the F1-score. It is defined as:

$$\text{Micro-F1} = \frac{2 \times \sum TP}{2 \times \sum TP + \sum FP + \sum FN}$$

where $\sum TP$, $\sum FP$, and $\sum FN$ denote the total number of true positives, false positives, and false negatives over all classes.

Chapter 4

Experimental Results

4.1 Reproduction of GCN Results

Table 4.1: Comparison between the original GCN results and our implementation

	GCN1 (%)			GCN2(%)		
Dataset	**acc**	**f1-micro**	**f1-macro**	**acc**	**f1-micro**	**f1-macro**
Cora	81.50	–	–	81.60	81.60	80.63
Citeseer	70.30	–	–	71.70	71.70	68.96
PubMed	79.00	–	–	78.60	78.60	78.21
Cornell	–	–	–	78.20	78.20	46.14
Texas	–	–	–	70.83	70.83	39.35
Wisconsin	–	–	–	66.36	66.36	54.42
Chameleon	–	–	–	48.94	48.94	47.10
				(46.84)	(46.84)	(46.26)
Squirrel	–	–	–	32.30	32.30	31.66
				(31.96)	(31.96)	(31.14)
Actor	–	–	–	36.81	36.81	33.05
Roman-Empire	–	–	–	25.59	25.59	15.29
Amazon-Ratings	–	–	–	38.17	38.17	17.80
Minesweeper	–	–	–	80.13	80.13	45.91
Tolokers	–	–	–	78.20	78.20	46.14
Questions	–	–	–	97.03	97.03	49.25

GCN1: results reported by Kipf and Welling (2017).
GCN2: results obtained from our implementation.
Values shown in parentheses for Chameleon and Squirrel correspond to the performance obtained after removing near-duplicate nodes.

The results presented in Table 4.1 show that the accuracy obtained by our implementation on the citation network datasets (Cora, Citeseer, and PubMed) is very close to the values reported in the original paper, confirming the consistency of our implementation. The Macro-F1 scores highlight the impact of class imbalance in some datasets, particularly Questions, Tolokers, Minesweeper, Cornell, and Texas, where the lower Macro-F1 compared to accuracy indicates unequal class distributions. Furthermore, after removing near-duplicate nodes from the Chameleon and Squirrel datasets, a decrease in all evaluation metrics is

observed, suggesting that duplicated nodes contributed to an overestimation of the original model performance.

4.2 Reproduction of Graphsage Results

Table 4.2: Comparison between the original GraphSAGE results and our implementation using different aggregation functions.

Dataset	Agg	GraphSAGE1 (%) f1-micro	GraphSAGE1 (%) f1-macro	GraphSAGE2 (%) f1-micro	GraphSAGE2 (%) f1-macro
PPI	GCN	50.00	–	51.13	30.33
	Mean	59.80	–	57.62	41.19
	LSTM	61.20	–	60.29	45.40
	Pool	60.00	–	59.11	43.35
Cora	GCN	–	–	81.40	80.12
	Mean	–	–	83.20	82.54
	LSTM	–	–	80.20	78.24
	Pool	–	–	81.20	80.85
Citeseer	GCN	–	–	73.00	67.25
	Mean	–	–	70.00	64.74
	LSTM	–	–	68.80	63.77
	Pool	–	–	69.00	63.85
PubMed	GCN	–	–	86.40	85.58
	Mean	–	–	90.00	89.78
	LSTM	–	–	89.60	89.14
	Pool	–	–	88.80	88.37
Cornell	GCN	–	–	67.80	49.87
	Mean	–	–	71.19	52.16
	LSTM	–	–	77.97	57.39
	Pool	–	–	64.41	48.33
Texas	GCN	–	–	64.41	35.55
	Mean	–	–	79.66	48.23
	LSTM	–	–	81.36	56.97
	Pool	–	–	66.10	38.85
Wisconsin	GCN	–	–	61.25	35.53
	Mean	–	–	75.00	44.05
	LSTM	–	–	77.50	48.37
	Pool	–	–	55.00	28.28
Actor	GCN	–	–	27.84	23.80
	Mean	–	–	33.43	30.70
	LSTM	–	–	33.35	30.49
	Pool	–	–	34.05	28.43
Roman-Empire	GCN	–	–	44.84	35.23
	Mean	–	–	71.24	62.88
	LSTM	–	–	71.37	63.60
	Pool	–	–	73.65	67.20
Amazon-Ratings	GCN	–	–	43.43	34.44
	Mean	–	–	45.47	36.94
	LSTM	–	–	43.21	35.54
	Pool	–	–	42.58	27.33
Minesweeper	GCN	–	–	80.00	44.44
	Mean	–	–	81.48	65.86
	LSTM	–	–	81.60	64.00
	Pool	–	–	82.04	65.79
Tolokers	GCN	–	–	77.82	46.20
	Mean	–	–	78.53	57.31
	LSTM	–	–	78.22	52.70
	Pool	–	–	78.26	44.36

Dataset	Agg	GraphSAGE1 (%) f1-micro	GraphSAGE1 (%) f1-macro	GraphSAGE2 (%) f1-micro	GraphSAGE2 (%) f1-macro
Questions	GCN	–	–	97.05	53.32
	Mean	–	–	96.25	58.42
	LSTM	–	–	96.35	58.67
	Pool	–	–	96.84	56.87
Chameleon	GCN	–	–	40.60	40.68
				(47.78)	(46.14)
	Mean	–	–	52.81	52.48
				(51.78)	(50.00)
	LSTM	–	–	49.38	47.99
				(55.92)	(55.47)
	Pool	–	–	42.94	41.83
				(46.89)	(44.84)
Squirrel	GCN	–	–	32.81	30.23
				(34.84)	(33.58)
	Mean	–	–	37.20	36.77
				(37.61)	(37.45)
	LSTM	–	–	34.38	33.77
				(35.74)	(35.33)
	Pool	–	–	35.64	33.87
				(38.76)	(36.40)

GraphSAGE1: results reported by Hamilton et al. (2017). Only the PPI dataset is available in the original paper for comparison.
GraphSAGE2: results obtained from our implementation using four aggregation functions (GCN, Mean, LSTM, and Pool).
Values in parentheses for Chameleon and Squirrel correspond to the results obtained after removing near-duplicate nodes.

Table 4.2 shows that:

- Our reproduction on the PPI dataset achieves results that are consistent with those reported in the original GraphSAGE paper.
- The four GraphSAGE aggregation variants (GCN, Mean, LSTM, and Pool) exhibit comparable overall performance, although the best-performing aggregator varies across datasets.
- Unlike GCN, removing near-duplicate nodes from the Chameleon and Squirrel datasets improves both the Micro-F1 and Macro-F1 scores for all GraphSAGE variants.

4.3 Reproduction of GAT Results

Table 4.3: Comparison between the original GAT results and our implementation

	GAT1 (%)			GAT2 (%)		
Dataset	**acc**	**f1-micro**	**f1-macro**	**acc**	**f1-micro**	**f1-macro**
Cora	83.00	–	–	83.40	83.40	82.35
Citeseer	72.50	–	–	73.20	73.20	67.38
PubMed	79.00	–	–	–	–	–
Cornell	–	–	–	45.40	45.40	15.37
Texas	–	–	–	55.36	55.36	14.25
Wisconsin	–	–	–	52.99	52.99	20.61
Chameleon	–	–	–	57.40	57.40	55.53
				(57.20)	(57.20)	(55.45)
Squirrel	–	–	–	36.03	36.03	31.05
				(36.95)	(36.95)	(35.56)
Actor	–	–	–	26.96	26.96	13.59

GAT1: results reported by Veličković et al. (2018).
GAT2: results obtained from our implementation.
Values shown in parentheses for Chameleon and Squirrel correspond to the performance obtained after removing near-duplicate nodes.

Table 4.3 shows that:

- Our results on Cora and Citeseer are consistent with those reported in the original GAT paper.
- Compared with GCN and GraphSAGE, duplicate removal improves the metrics on the Squirrel dataset, while it slightly decreases the performance on Chameleon.

4.4 Reproduction of GIN Results

Table 4.4: Comparison between the original GIN results and our implementation using 10-fold cross-validation. Results correspond to $\epsilon = 0$.

	GIN1 (%)	**GIN2 (%)**		
Dataset	**Accuracy**	**Accuracy**	**F1-micro**	**F1-macro**
MUTAG	89.4 ± 5.6	93.04 ± 4.90	93.04 ± 4.90	92.39 ± 5.17
PTC	64.6 ± 7.0	72.08 ± 4.93	72.08 ± 4.93	69.69 ± 6.83
NCI1	82.7 ± 1.7	78.03 ± 1.48	78.03 ± 1.48	78.01 ± 1.48
PROTEINS	76.2 ± 2.8	79.51 ± 3.31	79.51 ± 3.31	78.49 ± 2.94
IMDBBINARY	75.1 ± 5.1	74.80 ± 2.44	74.80 ± 2.44	74.59 ± 2.29
IMDBMULTI	52.3 ± 2.8	50.73 ± 1.59	50.73 ± 1.59	48.57 ± 2.84
REDDITBINARY	92.4 ± 2.5	78.20 ± 1.95	78.20 ± 1.95	78.15 ± 1.96
COLLAB	80.2 ± 1.9	– ± –	– ± –	– ± –
REDDIT-MULTI5K	57.5 ± 1.5	– ± –	– ± –	– ± –

GIN1: results reported by Xu et al. (2019) using GIN-ϵ with $\epsilon = 0$.
GIN2: results obtained from our implementation using 10-fold cross-validation.
Reported values correspond to the mean performance ± standard deviation over the folds.

The results obtained from our GIN implementation are generally consistent with those reported in the original paper (Xu et al., 2019), particularly on the IMDB-B and IMDB-M datasets, where we observe very close performance to both GIN-0 and GIN-ϵ variants. This indicates that our reproduction correctly captures the main behavior of the model.

However, some noticeable differences can be observed on certain datasets such as REDDIT-B and REDDIT-M5K, where our implementation shows lower performance compared to the reported results. These gaps may be attributed to implementation details, training setup, or hyperparameter sensitivity, which are known to significantly affect GIN performance on large-scale graph classification tasks.

Overall, these results confirm that our implementation is reliable while also highlighting the sensitivity of GIN to dataset characteristics and experimental configurations.

Bibliography

[1] T. N. Kipf and M. Welling, "Semi-Supervised Classification with Graph Convolutional Networks," Feb. 2017, arXiv:1609.02907 [cs.LG]. [Online]. Available: http://arxiv.org/abs/1609.02907

[2] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, "Graph attention networks," in *International conference on learning representations*, vol. 6, no. 2. Ithaca, 2018. [Online]. Available: https://www.chapterpal.com/s/vq6tnqa4/graph-attention-networks

[3] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How Powerful are Graph Neural Networks?" Feb. 2019, arXiv:1810.00826 [cs.LG]. [Online]. Available: http://arxiv.org/abs/1810.00826